

# Trapdoor Computing:

## Computing on Opaque Encodings

Alexander Towell  
lex@metafunctor.com

2026

### Abstract

Imagine a service that cannot read your queries and cannot read its own answers, yet returns the right ones. We make that precise. *Trapdoor computing* is a paradigm in which a *trusted* machine holds an encoder and a decoder while an *untrusted* machine sees only total functions on opaque bit strings flowing through opaque lookup tables. The untrusted machine cannot tell a real query from filler, one encoding from another, or a correct result from noise; its privacy comes from a one-way hash and a uniform representation, not from hiding access patterns. This is deliberately not oblivious RAM, not fully homomorphic encryption, and not a simulation-based security game. Confidentiality here is *measured* in the sense of quantitative information flow, an explicit leakage quantity a designer can compute and trade against space, bandwidth, and accuracy, rather than assumed *negligible* in the sense of a cryptographic reduction. This paper states the paradigm, draws its boundary against the cryptographic tools it is not, and maps the body of work that develops it: an abstraction, an algebra, a confidentiality theory, and concrete constructions.

## 1 The Oblivious Service

Consider a server that answers membership queries against a private set: a watchlist, a revocation list, an index. We want it to answer correctly while learning nothing it could turn against us, not which name was asked, not which names are on the list, not even how often a given name is asked. The standard cryptographic tools can do versions of this at a price: oblivious RAM hides the access pattern at a polylogarithmic bandwidth cost per access; fully homomorphic encryption computes on ciphertexts at a large constant and circuit-depth cost; searchable symmetric encryption is fast but leaks a documented profile that inference attacks exploit. Each makes a different trade, and each is the right tool for some regime.

Trapdoor computing occupies a different point. A *trusted* machine  $T$  holds a secret and the operations to encode an input and decode an output. An *untrusted* machine  $U$  holds a single object, a total function  $\hat{f}$  on bit strings, and evaluates it on whatever it is given. To answer a query,  $T$  encodes the input as a token  $c$ ,  $U$  returns  $\hat{f}(c)$ , and  $T$  decodes the result. The untrusted machine never holds the secret, never decodes, and never sees a value, only opaque tokens flowing through an opaque table. The privacy is not access-pattern hiding; it is that the tokens carry no readable signal.

## 2 The Paradigm

The organizing object is the *cipher map*: a total function  $\hat{f} : \{0, 1\}^n \rightarrow \{0, 1\}^n$  that implements a trapdoor approximation of a latent function  $f : X \rightarrow Y$ , paired with an encoder  $\text{enc}$ , a decoder  $\text{dec}$ , and a secret  $s$  from which all three derive [8]. The trusted machine computes  $\text{dec}(\hat{f}(\text{enc}(x)))$  and reads  $f(x)$ ; the untrusted machine computes  $\hat{f}$  on tokens and reads nothing. Four features of the definition do the work.

**Totality.**  $\hat{f}$  is defined on *all* of  $\{0, 1\}^n$ , not only on the tokens that encode real inputs. Hand it a string that encodes nothing and it still returns an  $n$ -bit answer. There is no “not found,” no exception, no out-of-band signal. This is the first privacy guarantee: with no special case to observe, the untrusted machine cannot separate meaningful queries from filler.

**Representation uniformity.** A value may be encoded as any of several tokens, and the encoding is chosen so that the stream of tokens the untrusted machine sees is close, in total variation, to uniform on the populated support. Frequent values are spread across more representations, classical homophonic substitution, so that no token is queried often and the traffic betrays nothing about which value was asked.

**Correctness.** The map is correct up to an error budget  $\eta$ : it computes  $f$  on in-domain inputs with probability at least  $1 - \eta$ . A nonzero  $\eta$  is not only a construction cost. It is also privacy: a reported result may be a true answer or a cipher-map error, and the ambiguity is deniability the untrusted machine cannot resolve.

**Composability.** Cipher maps chain. The output token of one is a well-formed input to the next, and the error of a chain is bounded predictably by the errors of its links. This is what makes trapdoor computing a calculus rather than a collection of isolated lookups.

**The four cannots.** Against the untrusted machine, these properties guarantee four things it cannot do, each blocked by a named property: it cannot *decode* a token to its value (one-way hash); it cannot *distinguish a real query from filler* (totality); it cannot *determine the domain* or read which value a token carries (representation uniformity); and it cannot *be sure a result is correct* (nonzero  $\eta$ ). These are the paradigm’s promises, and the rest of the program makes each one quantitative.

## 3 What Trapdoor Computing Is Not

The paradigm is easy to mistake for a neighbor, and the mistakes matter because they import the wrong threat model. Privacy here comes from one-way hashing and uniform representation, *not* from hiding access patterns; any argument that rests on access-pattern indistinguishability has drifted into the wrong framework. Table 1 draws the boundary.

The framework is also not a simulation-based or game-based cryptographic construction. It makes no claim that an adversary’s advantage is negligible in a security parameter. Its guarantees are leakage quantities, which is the next point and the heart of the paradigm.

Table 1: Trapdoor computing against its neighbors, by the difference in threat model, not a claim of superiority.

| Tool                       | Hides by                                   | Trapdoor computing instead                   |
|----------------------------|--------------------------------------------|----------------------------------------------|
| Oblivious RAM [4]          | access-pattern indistinguishability        | opaque tokens; access pattern not the secret |
| Fully homomorphic enc. [3] | computing on ciphertext                    | a hash table, not a circuit; bounded error   |
| Garbled circuits           | per-evaluation garbling                    | a reusable total function on tokens          |
| SSE / PPE [2, 5]           | a negligible-or-documented leakage profile | a <i>measured</i> , tunable leakage          |

## 4 Measured, Not Negligible

Cryptography’s strongest tools promise that what an adversary learns is *negligible*: a function vanishing faster than any inverse polynomial in a security parameter. That promise is the right one when it can be kept at an affordable cost. Trapdoor computing makes a different and more modest promise, borrowed from quantitative information flow [6, 1]: what the adversary learns is a *measured* quantity, a number a designer can compute and trade.

Concretely, the leakage of a single token is captured by an entropy ratio  $e = H(Q)/H^*$ , where  $H^* = \log_2 |\text{im}(\text{enc})|$  is the maximum entropy on the populated support (the usable bits of a token) and  $H(Q)$  the entropy actually observed; the representation-uniformity parameter  $\delta$  lower-bounds it through a Fannes–Audenaert continuity inequality, image-relative and linear in  $\delta$  [10]. Small  $\delta$  buys proportional confidentiality, at a known cost in space or bandwidth. This is the sense in which the paradigm’s privacy is engineering: not a guarantee of nothing learned, but an explicit budget of how much, traded against the resources a system can spend.

The honesty cuts both ways. A measured guarantee is weaker than a negligible one, and a reviewer schooled in simulation-based definitions will note its absence. But for the systems that cannot afford oblivious RAM or homomorphic encryption, a service that cannot read its queries or answers, returns the right ones, and leaks an amount you can write down is a real and useful guarantee, and a more honest one than a negligibility claim a construction cannot actually support.

## 5 The Program

The paradigm has been developed into a body of work, of which this paper is the front door. The *abstraction* fixes the cipher map and its four measurable properties, and shows that a single device, the acceptance predicate, unifies the constructions and traces a space-accuracy duality [8]. The *algebra and types* ask which type constructors survive the trapdoor: products pass, sums hit a genuine impossibility (tag-hiding and untrusted dispatch are exclusive), and an orbit-closure bound measures what an active adversary learns by operating on tokens [7]. The *confidentiality theory* measures leakage at two scales that do not reduce to each other, a marginal scale governed by  $\delta$  and a compositional scale, the joint distribution recoverable from shared-token observations, that no per-map parameter can improve [10]. And the *constructions* realize

cipher maps concretely, including a linear-algebraic retrieval structure whose frequency-hiding is structural, holding at zero per-query cost where every online defense pays per access [9].

The synthesis of these into one treatment, with the open problems they raise, is a monograph in preparation. This paper’s contribution is the paradigm itself: the trusted/untrusted model, the cipher map and its four cannots, the boundary against the cryptographic neighbors, and the stance that confidentiality is measured rather than negligible.

## 6 Conclusion

Trapdoor computing is a narrow and honest idea: compute on values sealed behind a one-way trapdoor, let an untrusted machine evaluate opaque tokens through an opaque table, and measure, rather than assume away, what leaks. It is not the strongest privacy tool, and it does not try to be. It is the tool for the middle ground between plaintext and the heavy cryptographic machinery, where a designer wants a guarantee that is real, composable, and affordable, and is willing to read the number that says how much.

**Provenance.** The framework was begun in 2017 and developed through 2020, then set aside and recovered, re-derived, and carried further in 2024. It builds on the Bernoulli model of approximate data structures developed alongside it.

## References

- [1] Mário S. Alvim, Konstantinos Chatzikokolakis, Annabelle McIver, Carroll Morgan, Catuscia Palamidessi, and Geoffrey Smith. *The Science of Quantitative Information Flow*. Springer, 2020.
- [2] Reza Curtmola, Juan Garay, Seny Kamara, and Rafail Ostrovsky. Searchable symmetric encryption: improved definitions and efficient constructions. In *CCS*, 2006.
- [3] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *STOC*, 2009.
- [4] Oded Goldreich and Rafail Ostrovsky. Software protection and simulation on oblivious RAMs. *Journal of the ACM*, 43(3):431–473, 1996.
- [5] Muhammad Naveed, Seny Kamara, and Charles V. Wright. Inference attacks on property-preserving encrypted databases. In *CCS*, 2015.
- [6] Geoffrey Smith. On the foundations of quantitative information flow. In *FoSSaCS*, 2009.
- [7] Alexander Towell. Algebraic cipher types: Confidentiality trade-offs in type constructors over trapdoor computing. Zenodo, 2026.
- [8] Alexander Towell. Cipher maps: Total functions as trapdoor approximations. Manuscript, <https://github.com/queelius/paper-cipher-maps>, 2026.
- [9] Alexander Towell. Codec-controlled retrieval: Structural frequency-hiding from the non-member channel of xor retrieval. Manuscript, under review (ACM TOPS), 2026.
- [10] Alexander Towell. The entropy ratio: Quantitative confidentiality for trapdoor computing. Zenodo, 2026.